Class practical

6.Implement a Basic N-gram Model for Text Generation

```
import random
from collections import defaultdict

text = "the cat sat on the mat the cat ate food"

words = text.lower().split()
bigrams = defaultdict(list)

# Build the bigram model
for i in range(len(words) - 1):
    bigrams[words[i]].append(words[i + 1])

# Generate text
word = "the"
result = [word]

for i in range(10):
    if word not in bigrams:
        break
    word = random.choice(bigrams[word])
    result.append(word)

print("Generated Text:")
print(" ".join(result))
```

```
Generated Text:
the cat sat on the cat ate food
```

7. Part-of-Speech Tagging Using the NLTK Library

```
import nltk

# Download the required POS tagging resource
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

# Input text
text = "The quick brown fox jumps over the lazy dog."

# Tokenize the text without using punkt
words = text.split()

# Perform POS tagging
tags = nltk.pos_tag(words)

# Display the result
print("POS Tagging Result:")
for word, tag in tags:
    print(word, ":", tag)
```

```
----------------------
POS Tagging Result:
The : DT
quick : JJ
brown : NN
fox : NN
jumps : VBZ
over : IN
the : DT
lazy : JJ
dog. : NN
```

8. Implement a Simple Stochastic POS Tagging Algorithm

```
import random

# Word-tag probability dictionary
tag_prob = {
    "book": {"NN": 0.8, "VB": 0.2},
    "read": {"VB": 0.9, "NN": 0.1},
    "the": {"DT": 1.0},
    "dog": {"NN": 1.0},
    "runs": {"VB": 0.8, "NNS": 0.2}
}

# Input sentence
sentence = "the dog runs"

words = sentence.split()
result = []

# Assign tags based on probabilities
for word in words:
    if word in tag_prob:
        tags = list(tag_prob[word].keys())
        probabilities = list(tag_prob[word].values())
        tag = random.choices(tags, weights=probabilities)[0]
    else:
        tag = "NN"

    result.append((word, tag))

print("Stochastic POS Tags:")
print(result)
```

```
Stochastic POS Tags:
[('the', 'DT'), ('dog', 'NN'), ('runs', 'NNS')]
```

9. Implement a Rule-Based POS Tagging System Using Regular Expressions

```
import nltk
from nltk import RegexpTagger

# Define regular expression rules
patterns = [
    (r'.*ing$', 'VBG'),
    (r'.*ed$', 'VBD'),
    (r'.*ly$', 'RB'),
    (r'.*ous$', 'JJ'),
    (r'.*', 'NN')
]

# Create the tagger
tagger = RegexpTagger(patterns)

# Input text
text = "The boy is running quickly"

# Tokenize the text without word_tokenize()
words = text.split()

# Perform POS tagging
tags = tagger.tag(words)

# Display the result
print("Rule-Based POS Tagging:")
for word, tag in tags:
    print(word, ":", tag)
```

```
Rule-Based POS Tagging:
The : NN
boy : NN
is : NN
running : VBG
quickly : RB
```

10. Implement Transformation-Based Tagging Using Transformation Rules

```
# Initial POS tags
sentence = [
    ("I", "PRP"),
    ("like", "NN"),
    ("playing", "NN"),
    ("football", "NN")
]

print("Before Transformation:")
print(sentence)

# Transformation rule:
# If a word is "like", change its tag from NN to VB.
for i in range(len(sentence)):
    word, tag = sentence[i]

    if word == "like" and tag == "NN":
        sentence[i] = (word, "VB")

# Transformation rule:
# Words ending in "ing" are tagged as VBG.
for i in range(len(sentence)):
    word, tag = sentence[i]

    if word.endswith("ing") and tag == "NN":
        sentence[i] = (word, "VBG")

print("\nAfter Transformation:")
print(sentence)
```

```
Before Transformation:
[('I', 'PRP'), ('like', 'NN'), ('playing', 'NN'), ('football', 'NN')]
```